

Data-Driven Decision Making in SQL

Case study:

1. Which is the favorite movie on MovieNow? Answer this question for a specific group of customers: for all customers born in the 70s.

Answer:

```
1 SELECT m.title,  
2 COUNT(*),  
3 AVG(r.rating)  
4 FROM renting AS r  
5 LEFT JOIN customers AS c  
6 ON c.customer_id = r.customer_id  
7 LEFT JOIN movies AS m  
8 ON m.movie_id = r.movie_id  
9 WHERE c.date_of_birth BETWEEN '1970-01-01' AND '1979-12-31'  
10 GROUP BY m.title  
11 HAVING COUNT(*) > 1 -- Remove movies with only one rental  
12 ORDER BY AVG(r.rating) DESC; -- Order with highest rating first
```

2. You're now going to explore actor popularity in Spain. Use as alias the first letter of the table, except for the table actsin use ai instead.

```
1 SELECT a.name, c.gender,  
2 COUNT(*) AS number_views,  
3 AVG(r.rating) AS avg_rating  
4 FROM renting as r  
5 LEFT JOIN customers AS c  
6 ON r.customer_id = c.customer_id  
7 LEFT JOIN actsin as ai  
8 ON r.movie_id = ai.movie_id  
9 LEFT JOIN actors as a  
10 ON ai.actor_id = a.actor_id  
11 WHERE country = 'Spain' -- Select only customers from Spain  
12 GROUP BY a.name, c.gender  
13 HAVING AVG(r.rating) IS NOT NULL  
14 AND COUNT(*) > 5  
15 ORDER BY avg_rating DESC, number_views DESC;
```

3. In chapter 1 you were asked to provide a report about the development of the company. This time you have to prepare a similar report with KPIs for each country separately. Your manager is interested in the total number of movie rentals, the average rating of all movies and the total revenue for each country since the beginning of 2019.

```

1 SELECT
2     c.country,                -- For each country report
3     COUNT(*) AS number_renting, -- The number of movie rentals
4     AVG(r.rating) AS average_rating, -- The average rating
5     SUM(m.renting_price) AS revenue -- The revenue from movie rentals
6 FROM renting AS r
7 LEFT JOIN customers AS c
8 ON c.customer_id = r.customer_id
9 LEFT JOIN movies AS m
10 ON m.movie_id = r.movie_id
11 WHERE date_renting >= '2019-01-01'
12 GROUP BY c.country;

```

4. Report a list of customers who frequently rent movies on MovieNow.

```

1 SELECT *
2 FROM customers
3 WHERE customer_id IN          -- Select all customers with more than 10 movie
                                rentals
4     [(SELECT customer_id
5        FROM renting
6        GROUP BY customer_id
7        HAVING COUNT(*) > 10)];

```

5. For the advertising campaign your manager also needs a list of popular movies with high ratings. Report a list of movies with rating above average.

query

```

1 SELECT title -- Report the movie titles of all movies with average rating higher than
               the total average
2 FROM movies
3 WHERE movie_id IN
4     (SELECT movie_id
5        FROM renting
6        GROUP BY movie_id
7        HAVING AVG(rating) >
8             (SELECT AVG(rating)
9              FROM renting));

```

6. Correlated query example:

A new advertising campaign is going to focus on customers who rented fewer than 5 movies. Use a correlated query to extract all customer information for the customers of interest.

```

1 -- Select customers with less than 5 movie rentals
2 SELECT *
3 FROM customers as c
4 WHERE 5 <
5     [(SELECT count(*)
6        FROM renting as r
7        WHERE r.customer_id = c.customer_id)];

```

7. Select all movies with an average rating higher than 8.

```

1 SELECT *
2 FROM movies AS m
3 WHERE 8 < -- Select all movies with an average rating higher than 8
4     (SELECT AVG(rating)
5      FROM renting AS r
6      WHERE r.movie_id = m.movie_id);

```

8. Query with exists

Having active customers is a key performance indicator for MovieNow. Make a list of customers who gave at least one rating.

```

1 SELECT *
2 FROM customers AS c -- Select all customers with at least one rating
3 WHERE EXISTS
4     (SELECT *
5      FROM renting AS r
6      WHERE rating IS NOT NULL
7      AND r.customer_id = c.customer_id);

```

9. In order to analyze the diversity of actors in comedies, first, report a list of actors who play in comedies and then, the number of actors for each nationality playing in comedies.

```

1 SELECT a.nationality, COUNT(*) -- Report the nationality and the number of actors for
   each nationality
2 FROM actors AS a
3 WHERE EXISTS
4     (SELECT ai.actor_id
5      FROM actsin AS ai
6      LEFT JOIN movies AS m
7      ON m.movie_id = ai.movie_id
8      WHERE m.genre = 'Comedy'
9      AND ai.actor_id = a.actor_id)
10 GROUP BY a.nationality;

```

10. The advertising team has a new focus. They want to draw the attention of the customers to dramas. Make a list of all movies that are in the drama genre and have an average rating higher than 9. This the example of “Query with exists and union , intersects”

```

1 SELECT *
2 FROM movies
3 WHERE movie_id in -- Select all movies of genre drama with average rating higher than
   9
4     (SELECT movie_id
5      FROM movies
6      WHERE genre = 'Drama'
7      INTERSECT
8      SELECT movie_id
9      FROM renting
10     GROUP BY movie_id
11     HAVING AVG(rating)>9);

```

11. CUBE

Prepare a table for a report about the national preferences of the customers from MovieNow comparing the average rating of movies across countries and genres.

```
1 SELECT
2     c.country,
3     m.genre,
4     AVG(r.rating) AS avg_rating -- Calculate the average rating
5 FROM renting AS r
6 LEFT JOIN movies AS m
7 ON m.movie_id = r.movie_id
8 LEFT JOIN customers AS c
9 ON r.customer_id = c.customer_id
10 GROUP BY CUBE (country, genre)
11 ORDER BY country; -- For all aggregation levels of country and genre
```

12. You are asked to study the preferences of genres across countries. Are there particular genres which are more popular in specific countries? Evaluate the preferences of customers by averaging their ratings and counting the number of movies rented from each genre.

```
1 -- Group by each country and genre with OLAP extension
2 SELECT
3     c.country,
4     m.genre,
5     AVG(r.rating) AS avg_rating,
6     COUNT(*) AS num_rating
7 FROM renting AS r
8 LEFT JOIN movies AS m
9 ON m.movie_id = r.movie_id
10 LEFT JOIN customers AS c
11 ON r.customer_id = c.customer_id
12 GROUP BY ROLLUP (c.country, m.genre)
13 ORDER BY c.country, m.genre;
```

13. You just saw that customers have no clear preference for more recent movies over older ones. Now the management considers investing money in movies of the best rated genres.

```

1 SELECT genre,
2        AVG(rating) AS avg_rating,
3        COUNT(rating) AS n_rating,
4        COUNT(*) AS n_rentals,
5        COUNT(DISTINCT m.movie_id) AS n_movies
6 FROM renting AS r
7 LEFT JOIN movies AS m
8 ON m.movie_id = r.movie_id
9 WHERE r.movie_id IN (
10      SELECT movie_id
11      FROM renting
12      GROUP BY movie_id
13      HAVING COUNT(rating) >= 3 )
14 AND r.date_renting >= '2018-01-01'
15 GROUP BY genre
16 ORDER BY avg_rating DESC; -- Order the table by decreasing

```

14. The last aspect you have to analyze are customer preferences for certain actors.

```

1 SELECT a.nationality,
2        a.gender,
3        AVG(r.rating) AS avg_rating,
4        COUNT(r.rating) AS n_rating,
5        COUNT(*) AS n_rentals,
6        COUNT(DISTINCT a.actor_id) AS n_actors
7 FROM renting AS r
8 LEFT JOIN actsin AS ai
9 ON ai.movie_id = r.movie_id
10 LEFT JOIN actors AS a
11 ON ai.actor_id = a.actor_id
12 WHERE r.movie_id IN (
13      SELECT movie_id
14      FROM renting
15      GROUP BY movie_id
16      HAVING COUNT(rating) >= 4)
17 AND r.date_renting >= '2018-04-01'
18 GROUP BY CUBE(nationality, gender); -- Provide results for all aggregation levels
    represented in a pivot table

```